NLP PROJECT: Sentiment Analysis for domain-specific texts

Pavlos Bozoglou | aivc24012

1. Introduction

Sentiment analysis (SA) has turned up to be one of the most active research topics in natural language processing and its significant applications are being used by a wide range of disciplines [1]. In its core, SA focuses on extracting and analysing information from data of people's expressions on various domains (e.g. products, topics, services), which is helpful to gain valuable insights [1] [2]. For this project we will work with SA for text classification in the hospitality domain (hotels, airlines, restaurants). In this domain, analysing customer feedback, or reviews, is essential and important for the managers to learn what the customers like, and it's also helpful to other customers to make informed decisions [1]. Of course, to effectively analyse customer reviews or other data its essential to handle domain-specific challenges. Some of the challenges in the hospitality domains include handling negation/irony, fake reviews, mixed sentiment, slang words, and other linguistic difficulties [1]. In this classification domain, most SA tasks follow a similar workflow (Figure 1), or steps, to handle these challenges and pre-process, model, and analyse the textual datasets, with different techniques and methods being used in each step based on the domain [1] [3]. The scope of this project is to specifically apply SA on hotel reviews data and test a number of classification models at predicting whether a review is positive or negative. To achieve that, we will be using natural language processing and machine learning frameworks in Python, with more details in the following sections.

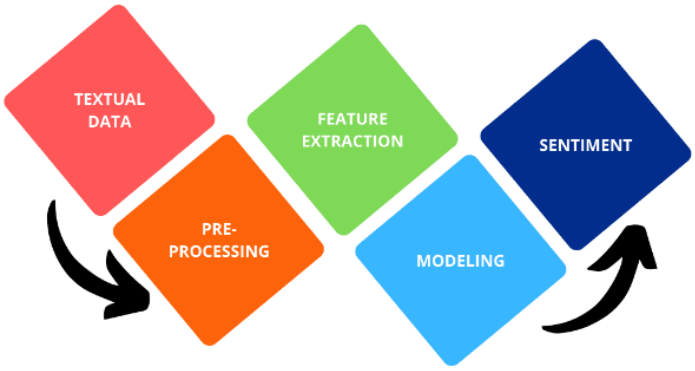

Figure 1 - Sentiment Analysis components for text classification [3].

2. Methodology

The main components of the methodology we applied consist of data collection, preprocessing, and model development, and we will cover each in the following subsections.

2.1 Data Collection & Exploration

The goal dataset is to get hotel reviews from the city of Athens. To achieve that we utilized the Google Places API, specifically the Nearby Search and Place Details API endpoints. The location used in the parameters of these endpoints were coordinates generated using Python and stored in a csv file with Pandas. The coordinates are evenly distributed to be approximately 0.5km from each other, covering the centre of Athens and are plotted in a map for visual verification (Figure 2 - Generated coordinates in the city area of Athens.).

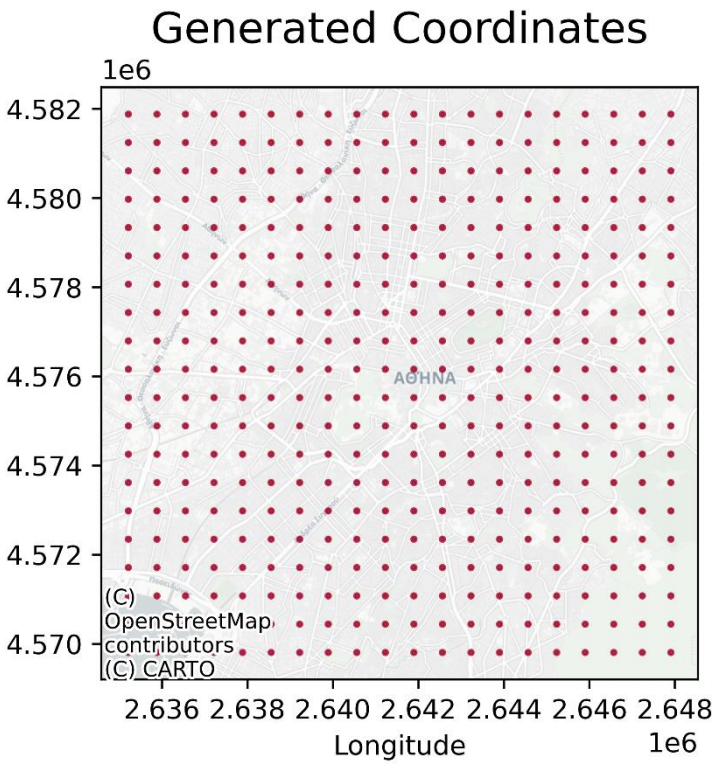

Figure 2 - Generated coordinates in the city area of Athens.

An API call was sent for each coordinate and nearby lodgings were collected with distance priority (closest). We ensured to skip any duplicates and respect the rate limits. The final dataset consists of 5623 reviews and stored in a csv file.

Following the data collection, we continued with an exploratory data analysis where we found that positive reviews amount to more than 80% of the dataset (- Dataset sentiment distribution before adjustment.). To fix that, we randomly dropped enough

positive reviews to evenly adjust and balance the dataset between positives and negatives (Figure 4).

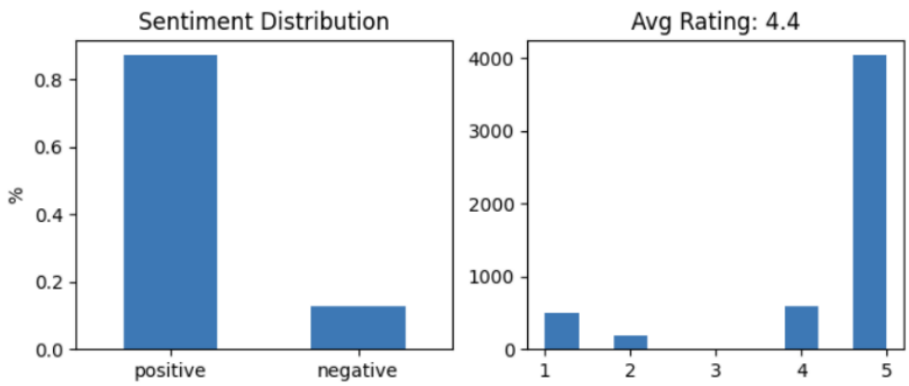

Figure 3 - Dataset sentiment distribution before adjustment.

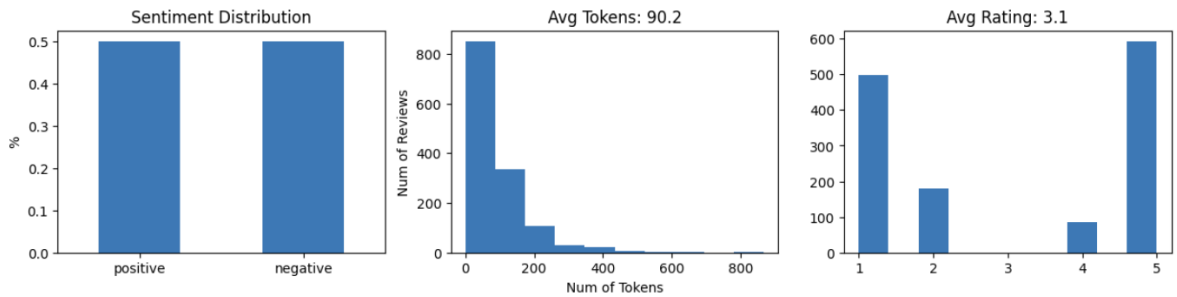

Figure 4 - Dataset sentiment and tokens distribution after adjustment.

2.2 Preprocessing

Reviews often have characters and words that we need to remove or edit before the modelling phase (Emoticons, URLs, Hashtags, Slang, etc). The method we used to clean the text can be seen in Figure 5, where we clean the emojis using the '*emoji*' library in python, then remove all punctuation and digits by applying the regular expression *'[^\w\s]|\d+]'* to each row, and finally we apply the '*word_tokenize*' function, the *'WordNetLemmatizer'* function to lemmatize each token and we only keep the word if it is in the word corpus and not in the stop words list of the NLTK framework.

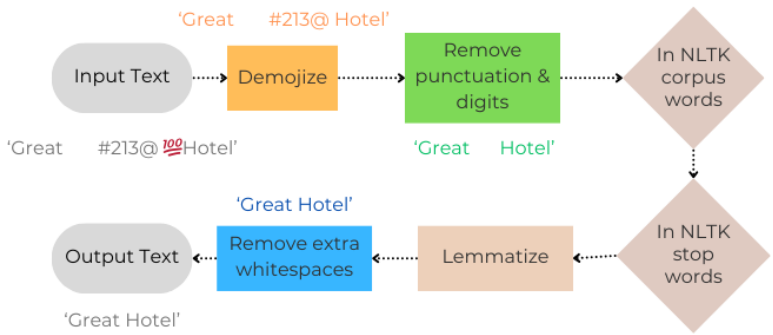

Figure 5 - Text preprocessing flowchart.

Due to the preprocessing, some reviews may now consist of an empty string, so we convert each review to a list of tokens and then drop any rows that have less than one token. Once the dataset is cleaned, the next step is to prepare it for modelling. To do that we need to split the dataset into train, testing, and labels. For the labels, we create a new column and encode the negative and positive labels to '0' and '1' respectively. For the train, test split we use the train_test_split function from the scikit-learn library with test_size=0,2. The final train data and test data ready for modelling have 1078 and 270 rows respectively.

2.3 Model development

We use several machine learning models to train and classify from the collected data, baseline models such as K-Nearest Neighbours (KNN), Decision Tree, Random Forest, Logistic Regression, Stochastic Gradient Descent (SGD), Multinomial Naive Bayes, Support Vector Classifier (SVC), and deep learning model such as Convolutional Neural Networks (CNN) and Long short-term Memory (LSTM) recurrent neural networks. For the baseline models we using the sklearn library to initialize and train them with their default parameters, except for the KNN model we set the n_neighbors parameter to 64 because it performed well after some testing. For the deep learning models we are using a maximum length of 60 tokens and trim/pad any exception. The deep learning architecture used are illustrated in the following Figures Figure 6, Figure 7.

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 60, 50)	50,000
lstm (LSTM)	(None, 128)	91,648
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 149,969 (585.82 KB)
Trainable params: 149,969 (585.82 KB)
Non-trainable params: 0 (0.00 B)

Figure 6 - LSTM model architecture.

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 60, 50)	50,000
conv1d (Conv1D)	(None, 58, 250)	37,750
dropout_2 (Dropout)	(None, 58, 250)	0
global_max_pooling1d (GlobalMaxPooling1D)	(None, 250)	0
dense_2 (Dense)	(None, 250)	62,750
dropout_3 (Dropout)	(None, 250)	0
dense_3 (Dense)	(None, 1)	251

Total params: 150,751 (588.87 KB)
Trainable params: 150,751 (588.87 KB)
Non-trainable params: 0 (0.00 B)

Figure 7 - CNN model architecture.

3. Evaluation & Performance Analysis

After the training, we evaluate the models with the test dataset and visualize the results. In Figure 8, we showcase the heatmap of the comparative performance analysis of all models, where the best and worst performing model seems to be the Naïve Bayes and the Decision Tree respectively, with most models having over 90% accuracy. Figure 9 displays a heatmap of the times needed to train each model measured in seconds and we can see that deep learning models take at least ten times the amount of train time compared baseline models, without any significant performance gain. Of course, this can differ in other datasets with larger amounts of data.

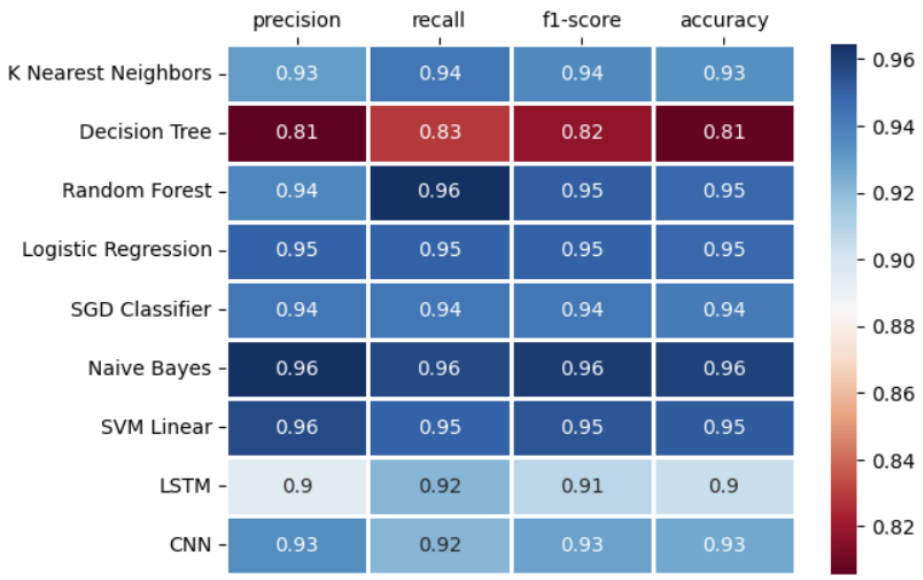

Figure 8 - Heatmap of all model performances.

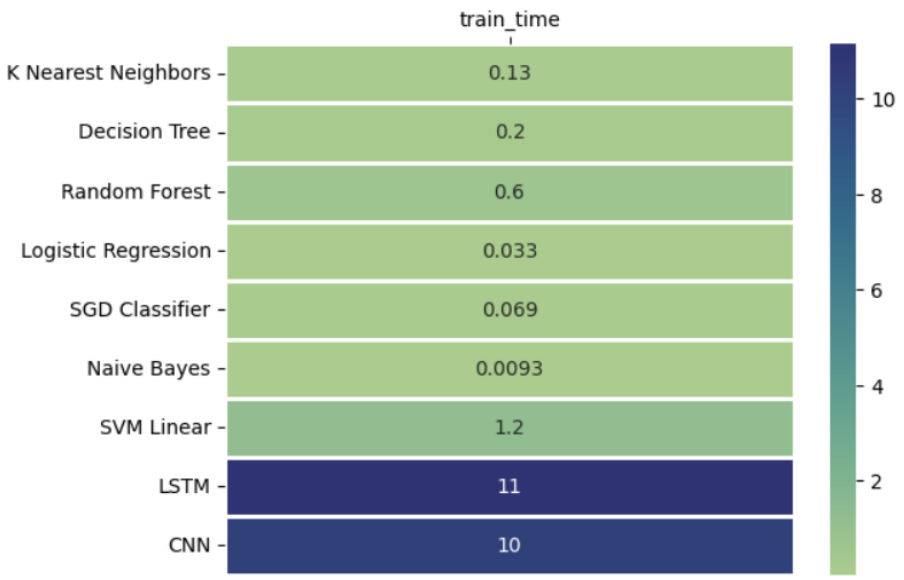

Figure 9 - Model train times in seconds.

In Figures 10, 11 we can observe the train accuracy and loss of the deep learning models, with both having a train accuracy of ~95% and a diverging validation accuracy which shows signs of potential overfitting if trained for more epochs.

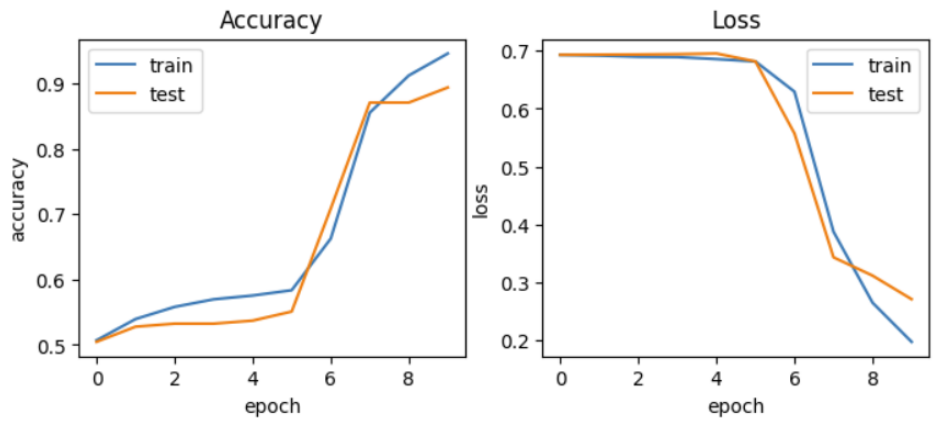

Figure 10 - LSTM model train accuracy and loss plots.

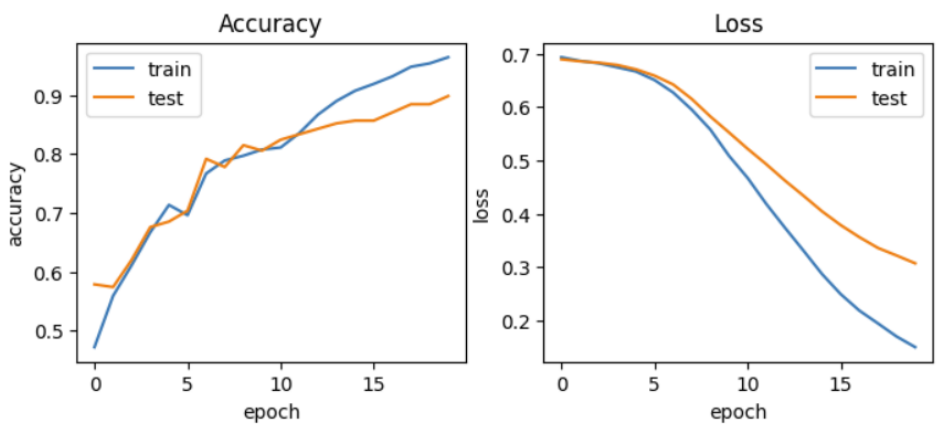

Figure 11 - CNN model train accuracy and loss plots.

Now that we have analysed and viewed the performance of all models, we can move to take a closer look at the samples that were classified/predicted a false label. Below in Figure 12 we see a bar plot of the misclassified samples from all the models, where the X axis represents the bin of each misclassified sample and the Y axis shows the number of models that misclassified the specific sample. Additionally, Table 1 provides a closer look at these samples.

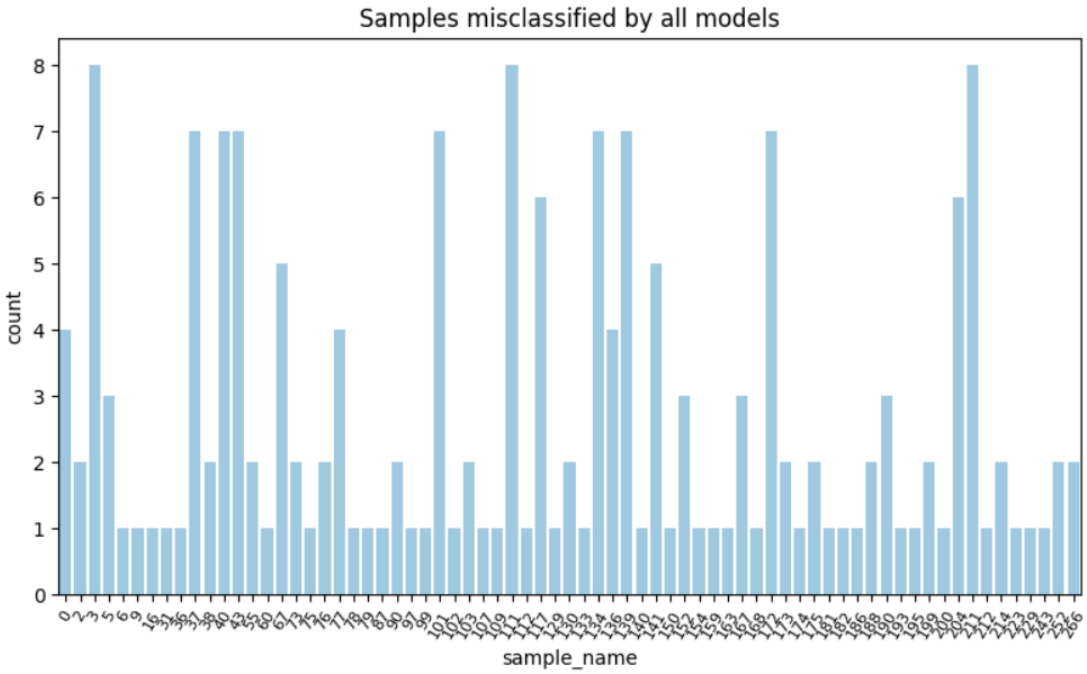

Figure 12 - Misclassified samples bar plot.

Table 1 - Texts of the top five misclassified samples.

Sample Name	Text	Misclassified by
139	"late night staff waiting u apartment away hotel reception fact taking u apartment not reception luckily clerk caught u elevator apartment building handle u luggage luckily set return bottom floor problem three sent luggage elevator room comfortable clean good great shower plenty hot water small kitchen cutlery morning head airport"	7/9 models
172	"bathroom level unclean unhygienic lot mould ceiling low price accommodation decent fine short stay nice addition acropolis also nice man received u check"	7/9 models
3	"lovely apartment didnt expect tea coffee close transport shopping busy street not used noise wasnt real issue expect entertainment area booking communication breeze"	8/9 models
111	"remarkable people respect"	8/9 models
211	"welcome car lady responsible way better doesnt reflect ethos hotel give bar staff ordered got first second drink wrong lovely rum returned got back right cork residue drink bar staff lovely helpful little matter level must visiting stay area"	8/9 models

To understand why these samples get misclassified, we extract the classifier weights and colour each word with it, in a pink-yellow-green, negative-neutral-positive respectively, colour space. The result of these method can be seen in Table 2 for the Logistic Regression model. We observe that the samples 139, 172, 111, and 211 confuse the model due to the large amount of neutral words that it learned to represent with negative/positive weight and it results in misclassification. On the other hand, sample 111 seems to have a very neutral colour, close to white, which indicates that the model is basically coin flipping and thus leads to a misclassification.

Table 2 - Text samples from the eyes of Logistic Regression.

Sample Name	Weight colored text	True Label	Prediction
139	"late night staff waiting u apartment away hotel reception fact taking u apartment not reception luckily clerk caught u elevator apartment building handle u luggage luckily set return bottom floor problem three sent luggage elevator room comfortable clean good great shower plenty hot water small kitchen cutlery morning"	Positive	Negative
172	"bathroom level unclean unhygienic lot mould ceiling low price accommodation decent fine short stay nice addition acropolis also nice man received"	Negative	Positive
3	"lovely apartment didnt expect tea coffee close transport shopping busy street not used noise wasnt real issue expect entertainment area booking"	Positive	Negative
111	"remarkable"	Positive	Negative
211	"welcome car lady responsible way better doesnt reflect ethos hotel give bar staff ordered got first second drink wrong lovely rum returned got back right cork residue drink bar staff lovely helpful little matter level must visiting"	Negative	Positive

Furthermore, by collecting the weights from all the models, we can take the top positive and negative words of each model and visualize it in a word cloud to gain insights

on which words the models perceive as positive and negative as seen in Figures Figure 13 – Word cloud of top positive words learned., Figure 14 – Word cloud of top negative words learned..

Figure 13 – Word cloud of top positive words learned.

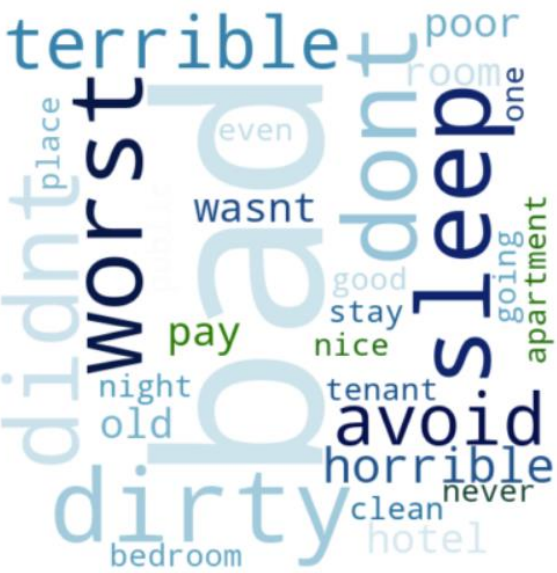

Figure 14 – Word cloud of top negative words learned.

Lastly, we create a map of Athens to visualize each lodging coordinate with a colour based on the results of the deep learning models and compare the predictions with the original true sentiment label.

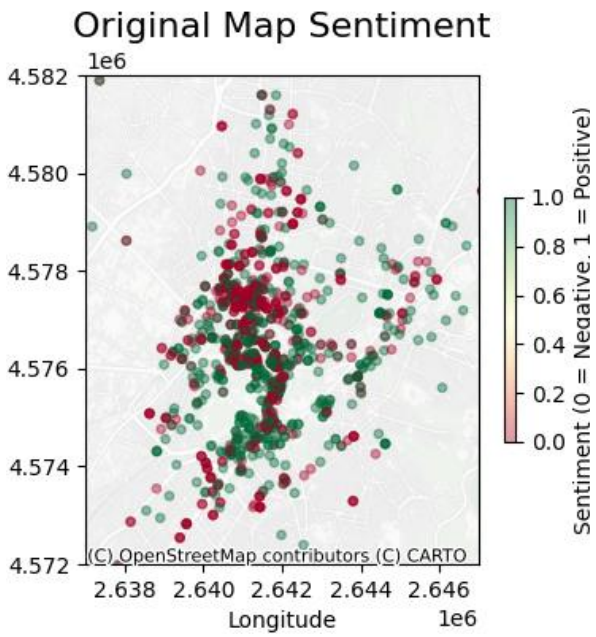

Figure 15 - Coloured coordinates using true labels.

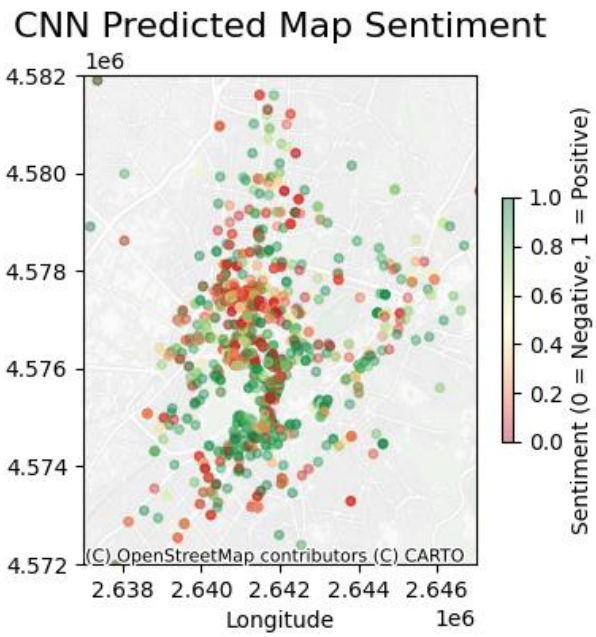

Figure 16 - Coloured coordinates using CNN model predictions.

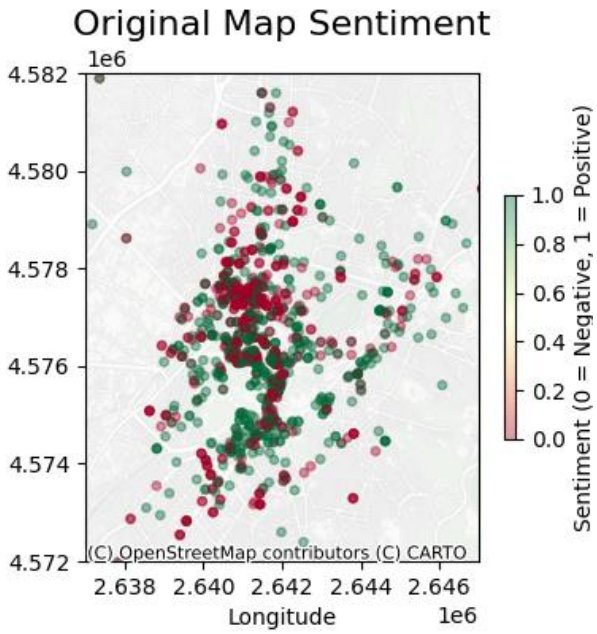

Figure 17 - Coloured coordinates using true labels.

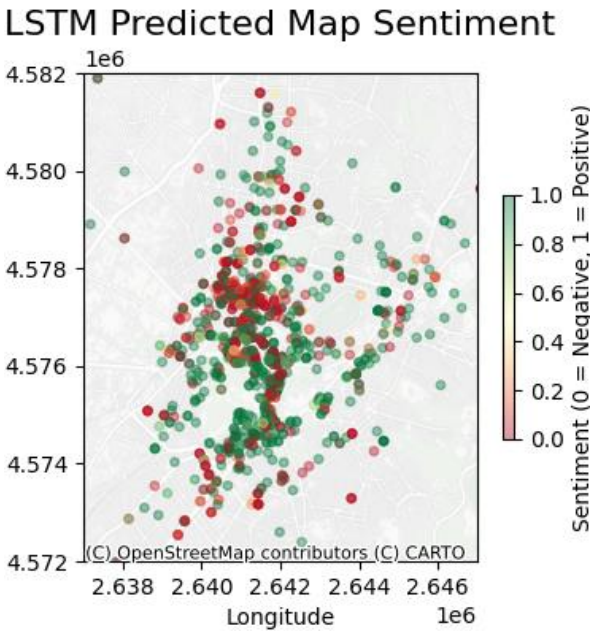

Figure 18 - Coloured coordinates using LSTM model predictions.

4. Conclusions

Applying sentiment analysis in textual data is a very valuable and can offer significant insights. The methodology we have used gave great results, with the models showing excellent performance at understanding even domain specific positive and negative words, such as acropolis, clean, dirty, mould. In the future, for a more professional and reliable results, a larger dataset is recommended in order to learn the features better. Additionally, changes in the methodology can be applied, such as methods to remove neutral words that confuse the models.

5. References

[1] A. Ameus, S. Hamdi and S. Ben Yahia, “Sentiment Analysis for Hotel Reviews: A Systematic Literature Review,” *ACM Computing Surveys,* vol. 56, no. 2, pp. 1-32, 2023.

[2] Wikipedia, “Sentiment analysis,” Wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/Sentiment_analysis. [Accessed 14 01 2025].

[3] K. Sudhanshu, R. Partha Pratim , D. Prosad Dogra and K. Byung-Gyu , “A Comprehensive Review on Sentiment Analysis: Tasks, Approaches and Applications,” arXiv, 2023.

[4] “Alibi Explain,” Seldon Technologies, [Online]. Available: https://docs.seldon.io/projects/alibi/en/stable/examples/integrated_gradients_imdb.html. [Accessed 03 01 2025].

[5] L. A. Wulandiari, “SENTIMENT ANALYSIS | HOTEL REVIEW [Python-NLTK],” [Online]. Available: https://medium.com/@amalia.wulandiari/sentiment-analysis-hotel-review-python-nltk-846738de727f. [Accessed 05 01 2025].

6. Appendix

The code python code and notebooks, along with the datasets, can be found in the following GitHub repository link: https://github.com/pbozoglou/Sentiment-Analysis-applied-in-the-hospitality-domain